OOM KILLER

CUANDO LINUX SE QUEDA SIN MEMORIA

Caso de estudio: arquitectura, heurística de decisión y comportamiento del kernel ante el agotamiento de memoria

Estudiante: **James Tack**
Profesor: **Isacio Tamayo**
Grupo: **1IL143**

Universidad Tecnológica de Panamá
Facultad de Ingeniería de Sistemas Computacionales
Departamento de Computación y Simulación de Sistemas

QUÉ VAMOS A VER

Estructurado sobre la guía de preguntas del caso de estudio

P1 ¿Qué significa que un proceso use memoria?

P2 ¿Qué ocurre cuando varios procesos piden más memoria?

P3 ¿Cómo sabe el kernel cuánta memoria hay disponible?

P4 ¿Qué es el OOM Killer?

P5 ¿Por qué el kernel decide terminar un proceso?

P6 ¿Qué consecuencias tiene terminar ese proceso?

★ Discusión: si fueran el kernel, ¿a quién matarían?

LA SITUACIÓN

Un sistema Linux tiene muy poca memoria disponible. Varios procesos continúan solicitando memoria hasta que el sistema alcanza una situación crítica y el kernel debe tomar una decisión para recuperar memoria.

- Este es el escenario exacto que vamos a diseccionar técnicamente, capa por capa.
- No lo vamos a dejar como una "caja negra": vamos a ver qué hace el kernel en cada paso, con la documentación oficial como fuente.
- La guía plantea 6 preguntas — las vamos a responder en orden, y cerramos con una discusión abierta.

¿QUÉ SIGNIFICA QUE UN PROCESO USE MEMORIA?

- Memoria virtual: el espacio de direcciones que el proceso "ve" — no es RAM física todavía.
- Memoria física (RSS): las páginas que realmente están cargadas en RAM en este momento.
- Un proceso usa memoria en varias formas: heap (malloc/new), stack, código y bibliotecas mapeadas, y mmap.
- Ninguna de esas páginas se vuelve física hasta el primer page fault — cuando el proceso realmente escribe o lee.

VIRTUAL ≠ FÍSICA

"Usar memoria" tiene dos capas: la promesa (espacio de direcciones) y el consumo real (páginas físicas residentes).

Se puede consultar en:
/proc/<pid>/status → VmSize (virtual)
vs VmRSS (física)

LINUX TE MIENTE SOBRE LA MEMORIA

- Cuando varios procesos piden memoria a la vez, el kernel no reserva RAM física de inmediato para cada uno.
- Las páginas físicas se asignan por demand paging — recén en el primer page fault de cada proceso.
- Como muchos procesos nunca usan todo lo que reservaron, el kernel permite overcommit: prometer más memoria de la que existe físicamente.
- Un sistema con 16GB RAM + 8GB swap puede tener 40GB comprometidos en el papel entre todos los procesos.

40GB

comprometidos entre todos los procesos, sobre solo 16GB de RAM física + 8GB de swap

`/proc/meminfo → CommitLimit vs Committed_AS`

TRES POLÍTICAS, UN INTERRUPTOR

`vm.overcommit_memory` — decide cómo se resuelve la contención entre procesos

0

Heurístico (default)

Rechaza asignaciones obviamente excesivas; permite el resto.

1

Siempre overcommit

Nunca rechaza. Útil en apps científicas con arrays dispersos.

2

Sin overcommit

Límite estricto: swap + % configurable de RAM física.

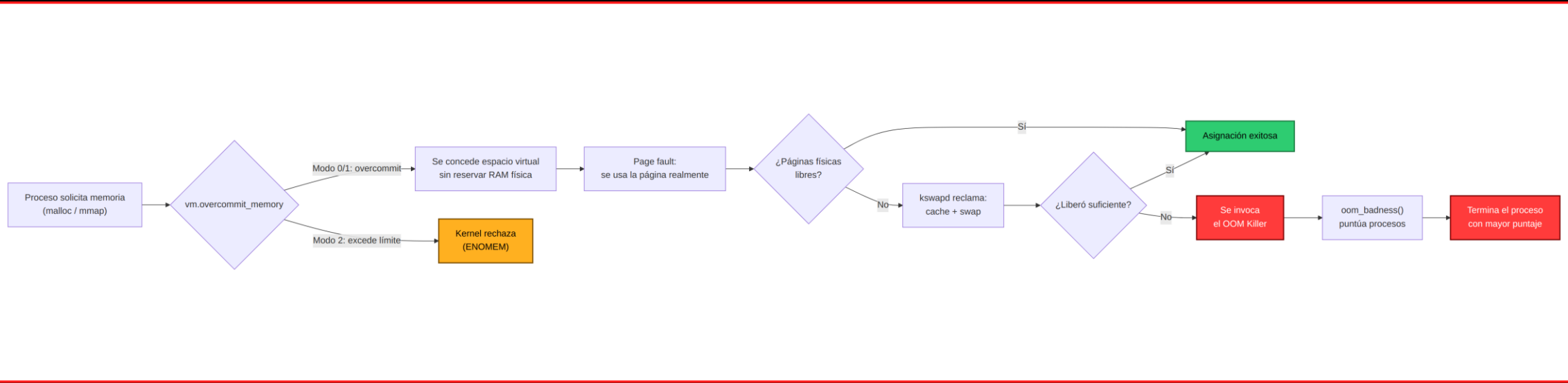
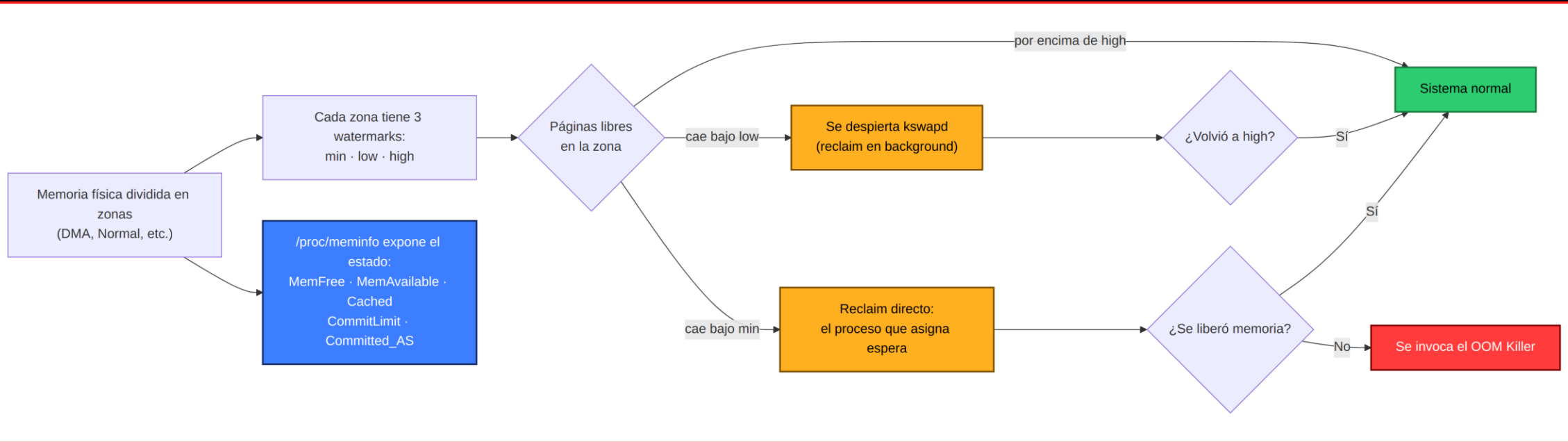


Diagrama · Del overcommit al OOM Killer: qué pasa cuando la promesa de memoria se convierte en demanda real.

CÓMO MIDE EL KERNEL LA MEMORIA DISPONIBLE



- La RAM física se divide en zonas; cada zona mantiene 3 marcas de agua: min, low y high.
- Bajo low, se despierta kswapd en background; bajo min, el proceso que asigna hace reclaim directo y espera.
- Todo esto se resume para el usuario en /proc/meminfo (MemFree, MemAvailable, Cached, Committed_AS).

PREGUNTA 4

¿QUÉ ES EL OOM KILLER?

Es un mecanismo del kernel de Linux (función oom_kill() en mm/oom_kill.c) que se activa cuando el sistema está críticamente bajo de memoria física y swap, y ya no queda nada más que reclamar. Su tarea: elegir un proceso y terminarlo para liberar memoria y mantener el sistema vivo.

- No es un antivirus ni un proceso separado: es parte del subsistema de gestión de memoria del kernel.
- No actúa por capricho — solo se invoca cuando kswapd y el reclaim directo ya fallaron.
- Es la última línea de defensa del kernel contra un colapso total del sistema.

PREGUNTA 5

¿POR QUÉ MATAR EN VEZ DE FALLAR?

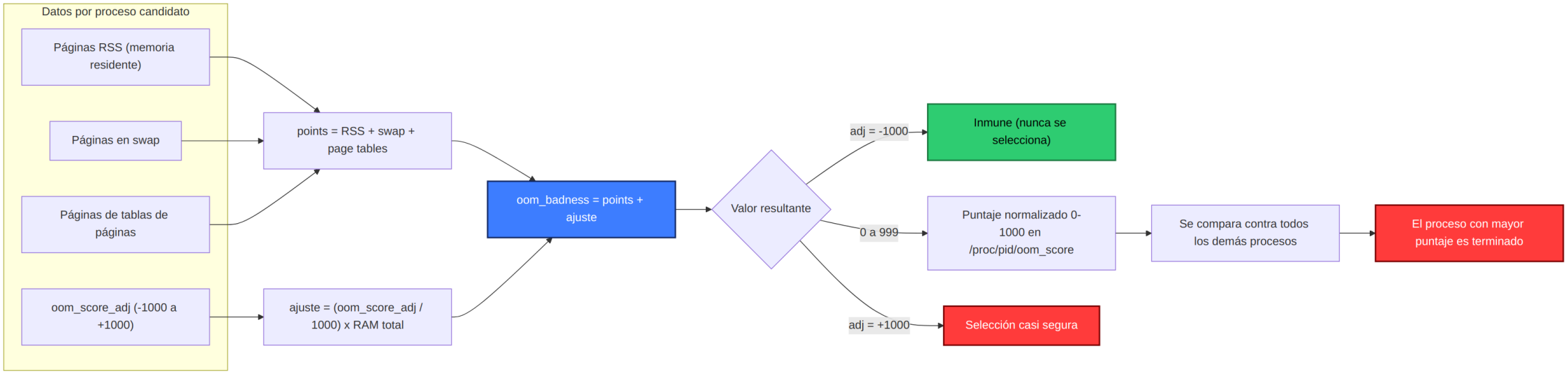
- Primero actúa kswapd: limpia page cache y escribe páginas anónimas sucias a swap.
- Solo si eso no basta, se invoca el OOM Killer — es el último recurso del kernel.
- Bajo overcommit, la memoria ya fue prometida a otro proceso distinto al que la necesita ahora.
- Gran parte del software en C nunca comprueba si malloc() devolvió NULL.
- Terminar un proceso específico es más seguro que arriesgar corrupción de datos en todo el sistema.

ÚLTIMO RECURSO

El OOM Killer nunca es la primera respuesta del kernel — es la última, cuando ya no queda nada más que reclamar.

CÓMO ELIGE: oom_badness()

mm/oom_kill.c · kernel de Linux



- Cada proceso candidato recibe un puntaje de 0 (nunca matar) a 1000 (candidato seguro).
- points = páginas RSS + páginas en swap + páginas de tablas de páginas.
- Se suma un ajuste basado en oom_score_adj × RAM total del sistema.
- El resultado se consulta en vivo en /proc/<pid>/oom_score.

PREGUNTA 5

AJUSTANDO QUIÉN SOBREVIVE



-1000 · INMUNE

+1000 · PRIMER OBJETIVO

- /proc/<pid>/oom_score_adj acepta valores de -1000 a +1000.
- systemd: OOMScoreAdjust=-1000 protege servicios críticos (ej. bases de datos).
- Principio del "sacrificio del hijo": el kernel prefiere matar primero a los hijos con espacio de memoria separado.

OOMScoreAdjust=-1000

Ejemplo real: unidad systemd protegiendo un proceso crítico

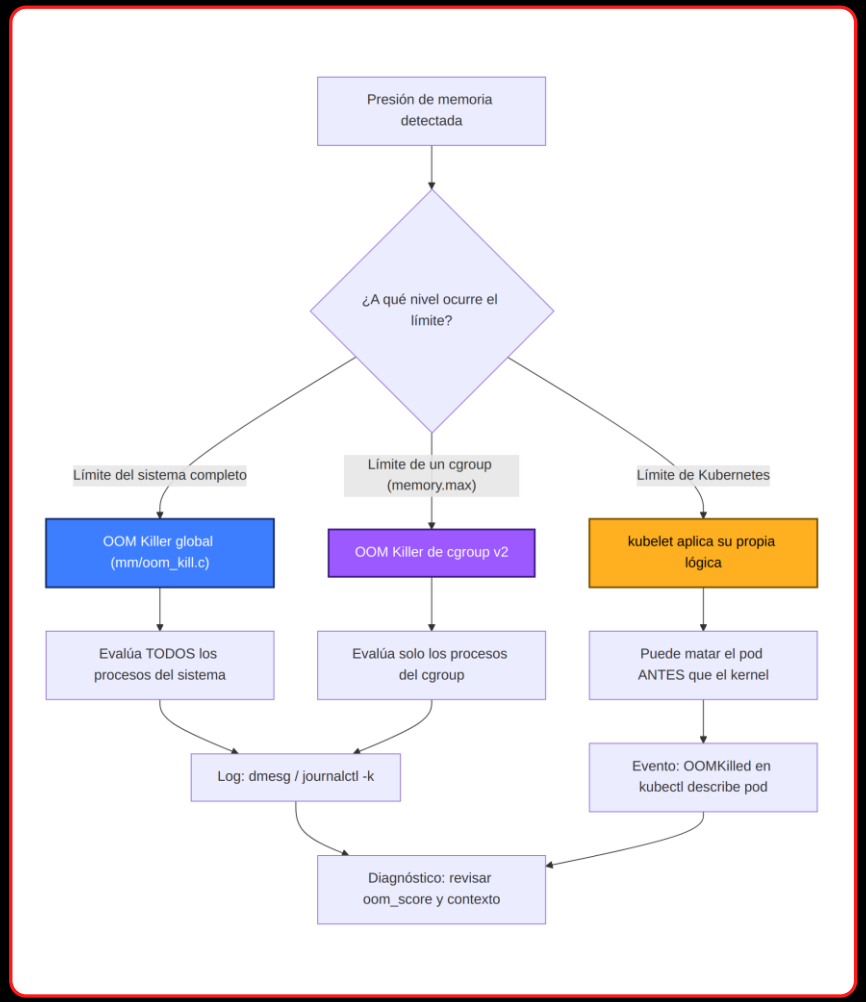
```
[Service]
ExecStart=/usr/bin/postgres
OOMScoreAdjust=-1000
```

¿QUÉ CONSECUENCIAS TIENE TERMINARLO?

SIGKILL inmediato	El proceso recibe la señal 9 — no puede capturarla, no hay limpieza ni cierre ordenado.
Pérdida de trabajo no guardado	Cualquier dato en memoria que no se había persistido a disco se pierde por completo.
Posible reinicio automático	systemd o Kubernetes pueden relanzar el servicio — pero hay downtime real mientras tanto.
Riesgo de elegir mal	Si el puntaje elige al proceso equivocado (ej. sshd), el diagnóstico se vuelve mucho más difícil.
Rastro en logs	Queda evidencia en dmesg/journalctl — clave para el post-mortem, pero no evita el impacto inicial.

OOM EN CGROUPS Y KUBERNETES

- cgroups v2: memory.max dispara un OOM local, limitado a los procesos de ese grupo.
- Kubernetes: el kubelet monitorea container_memory_working_set_bytes.
- El kubelet puede matar un pod ANTES de que el kernel active su propio OOM Killer.
- Diagnóstico: un pod puede morir por decisión del kernel o del orquestador — son cosas distintas.



- Evento visible: OOMKilled en kubectl describe pod.

PROFUNDIZANDO

EL KERNEL SIEMPRE DEJA RASTRO

- dmesg / journalctl -k → busca la línea "invoked oom-killer".
- El log incluye el proceso elegido y su oom_score en el momento del evento.
- Ranking en vivo: leer /proc/[pid]/oom_score de todos los procesos y ordenar de mayor a menor.
- panic_on_oom: en vez de matar un proceso, hace panic del sistema completo — usado en clusters que prefieren reinicio limpio a estado corrupto.

```
$ dmesg | grep -i oom-killer

kernel: postgres invoked
oom-killer: gfp_mask=..
out_of_memory: Killed
process 4211 (postgres)
```

PREGUNTA PARA DISCUSIÓN

SI USTEDES FUERAN EL KERNEL...

¿Qué proceso considerarían para terminar, y por qué? Miremos un sistema real bajo presión:

Proceso	RSS	Tiempo activo	oom_score_adj	Rol
`sshd`	8 MB	40 días	-1000	Acceso remoto al servidor
`chrome (pestaña)`	1.8 GB	6 min	0	Pestaña de usuario recién abierta
`postgres`	2.4 GB	40 días	0	Base de datos en producción
`backup_script.sh`	120 MB	2 min	+200	Job de respaldo nocturno
`log_analyzer`	3.1 GB	18 min	0	Proceso propio del estudiante, con fuga de memoria

Piensen en oom_badness(), en el impacto de cada consecuencia, y en qué haría un administrador antes de que el kernel tenga que decidir.

CIERRE

CONCLUSIONES

No es un bug

Es la consecuencia lógica y necesaria de permitir overcommit de memoria.

Es predecible

Se puede controlar con oom_score_adj y los límites de cgroup.

Tiene un costo real

SIGKILL, pérdida de trabajo no guardado y downtime — nunca es gratis.

El origen importa

En contenedores, distinguir si mató el kernel o el orquestador es clave para el diagnóstico.

¿PREGUNTAS?

GRACIAS

James Tack · Ingeniería en Sistemas y Computación · UTP